

Union statements don't scale. Your lakehouse metadata can.



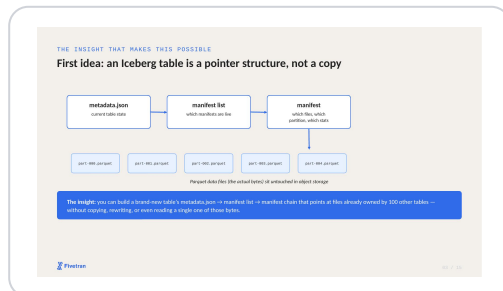
When the same table shape shows up across many connections — one per tenant, one per region, one per shard — the default fix is a **UNION ALL** view or a scheduled merge job. Both work. Both are also more expensive, and more fragile, than most teams realize.

THE WAY MOST TEAMS DO IT TODAY

- **Every query re-scans every source.** A UNION ALL view isn't a table — it's a live instruction to re-read all of your data, from every source, every single time someone runs a query against it.
- **Scheduled merge jobs don't avoid the cost — they just move it.** A nightly job that rewrites the consolidated table still reprocesses everything, whether or not anything actually changed since yesterday.
- **Schema drift breaks things quietly.** One source adds, renames, or retypes a column, and the union either fails outright or silently produces the wrong result — with no query-time warning.
- **Every new source is a maintenance event.** Adding tenant #101 means editing the view, the job, or both — by hand, indefinitely.

WITH MDLS + NO-REWRITE CONSOLIDATION

- **Zero data is copied or rescanned to consolidate.** MDLS stores your tables in the open Apache Iceberg format, where a table's real identity is its metadata — a manifest pointing at data files, not the files themselves.
- **Building the consolidated view is a metadata operation.** A new table's manifest can point directly at every source's existing files. No `SELECT`, no rewrite, no reprocessing — just a fast index update.
- **Schema drift is handled by design.** New columns widen the schema automatically; only a genuine conflict between two sources triggers a rewrite, and it's scoped to just the drifted rows — never the whole table.
- **Adding a source is operational, not computational.** Consolidation cost tracks how much changed, not how much data exists.



< 40 sec

to fully consolidate 100 identically-shaped tables across 3 sources into one queryable table — validated end-to-end against real Fivetran-managed infrastructure, zero bytes rewritten.

How it works, briefly

An Iceberg table's "data" is really just a pointer structure: [metadata.json](#) → [manifest list](#) → [manifest](#) → [Parquet files](#), which sit untouched in object storage the whole time. Think of it like a library card catalog: a **UNION ALL** view is like re-reading every relevant book from every branch, every time someone asks a question. This approach builds one new card catalog whose cards point directly at books already sitting on the shelves of every branch — nothing is copied, and nothing has to be reconstructed at read time.

The one place this gets genuinely tricky — and where the engineering effort actually goes — is that Iceberg identifies columns by an internal number, not by name. Two sources that evolve independently can land a new column on the same internal number with two different meanings, and Iceberg won't error on its own — it'll silently return the wrong data. This technique reserves a permanently safe numbering range up front and detects any real collision automatically, so the fast path stays fast and the rare edge case gets caught rather than glossed over.

0

bytes rewritten
in the common case

1

consolidated table,
any number of sources

Auto

schema drift
handling

0

static cloud keys
required to run it